

Lab2 CS2 – Backtracking

Problem: Magic Number Squares

In this lab, you will solve the magic number square problem using backtracking strategy.

Submission: In order to submit your code, you need to go to your main lecture section's webcourses and click Lab2 to access Codegrade. After completing the code, please submit it to Codegrade. Your file name has to be Main.java and the class name has to be Main as well.

Magic Square : $N \times N$ matrix containing numbers from 1 to N^2 where the sum of each row, column and both main diagonals are the same. The summed value is called magic constant. N determines the order of the magic squares.

There are magic squares of 3rd, 4th, 5th, 6th orders and so on.

For $N = 3$, magic constant = 15,

For $N = 4$, magic constant = 34,

For $N = 5$, magic constant = 65, etc.

Magic constant = $N(N^2 + 1)/2$

There are 8 different variations of magic square for 3x3 and 880 different magic squares for 4x4, 275,305,224 for 5th order and 1.8×10^9 for 6th order.

!! No known formula to find the magic squares for given order. !!

A 3×3 magic square is an arrangement of the numbers from 1 to 9 in a 3 by 3 grid, with each number occurring exactly once, and such that the sum of the entries of any row, any column, or any main diagonal is the same.

- For a 3x3 magic number square the sum is $3 \cdot (3^2 + 1)/2 = 15$.
-

2	7	6	→15
9	5	1	→15
4	3	8	→15

15 ↓ ↓ ↓ ↓ 15

It is possible to have different sized magic number squares of size $n \times n$, where numbers $\{1, 2, \dots, n \times n\}$ must be arranged in such a way that every row and column has the same sum.

◦ For a $n \times n$ magic number square the sum is $n \times (n \times n + 1) / 2$

Say we want to create a class MagicSquare that takes in an int n and returns all valid Magic Number Squares of size $n \times n$.

◦ Example, $n = 3$ returns 8 magic squares:

2 7 6 9 5 1 4 3 8	4 9 2 3 5 7 8 1 6	8 1 6 3 5 7 4 9 2
2 9 4 7 5 3 6 1 8	6 1 8 7 5 3 2 9 4	8 3 4 1 5 9 6 7 2
4 3 8 9 5 1 2 7 6	6 7 2 1 5 9 8 3 4	

Sample input:

3 (you can assume $n > 2$)

Sample output:

N = 3

Done!!

[2, 7, 6]

[9, 5, 1]

[4, 3, 8]

Done!!

[2, 9, 4]

[7, 5, 3]

[6, 1, 8]

Done!!

[4, 3, 8]

[9, 5, 1]

[2, 7, 6]

Done!!

[4, 9, 2]

[3, 5, 7]

[8, 1, 6]

Done!!

[6, 1, 8]

[7, 5, 3]

[2, 9, 4]

Done!!

[6, 7, 2]

[1, 5, 9]

[8, 3, 4]

Done!!

[8, 1, 6]

[3, 5, 7]

[4, 9, 2]

Done!!

[8, 3, 4]

[1, 5, 9]

[6, 7, 2]